

1 What I built, and what I did not

A fleet soiling advisor: where the truck goes tomorrow, and the work order that carries it out.

CSV

ingest

quality gate

soiling estimate

economics

snapshot

API

UI

Source tables mirror the CSVs and are never mutated. Everything downstream is derived, pure, and can be dropped and rebuilt.

Five decisions I would defend

1. A quality gate sits before the estimator

Withhold any plant-day under PR 0.5. Soiling accumulates, so a loss that reverses overnight without a wash is not one a wash recovers. The gate never names a cause.

Ungated, the formula ranks plant_1003 at +\$1,498,102 while it makes 2% of expected.

2. The brief's formula, with one input changed

s_0 is a state: today's gated reading, multiplied by nothing. E is a rate the formula multiplies by T, so it takes a 14-day median, not today's weather.

E swings 26% day over day — alone it flips plant_1005 from −\$4,164 to +\$1,692.

3. Region is a hard partition, not a column

Every crew's home base maps onto exactly one plant region, so there are five independent rankings and no fleet-wide one.

Pooling crews claims 6.9 crew-days against a real 10.5-day Rajasthan bottleneck.

4. One precomputed snapshot per day

The ranking is a pure function of data through yesterday and cannot change during the day, so ~120 viewers at 8am read one materialised answer.

5. The LLM writes prose, never numbers

Computed values in, briefings and work orders out. It never ranks, values or detects, and falls back to a deterministic template with no API key.

A dispatch freezes the numbers it was approved on — an audit record, not a pointer.

Today's answer

5 of 12	plants worth cleaning
\$44,719	recoverable
206 / 1,343	readings withheld

Deliberately not built

crew routing and travel time
rainfall forecasting
readings/ ingestion — 530 MB
multi-day job scheduling
auth, realtime, a crew app
None changes tomorrow's dispatch.

DEMO — two minutes

fleet table → plant_1000, break-even as arithmetic → dispatch
and work order → a withheld day, no diagnosis

2

The decision I am least sure about

How to model soiling as a function of time — and of the time the cleaning itself takes.

$$\text{soiling}(t) = s_0 + r \cdot t \quad \text{gain} = \int_0^T (s_0 + r \cdot t) dt - \int_0^T (r \cdot t) dt = s_0 \cdot T$$

Both trajectories accumulate at the same rate, so r cancels out of the value. Everything rests on s_0 and T — which is exactly why s_0 has to be right.

Two estimators built, measured, and deleted

Forward projection — $s_0 + r \cdot T/2$

I tuned against the forward mean soiling of the uncleaned plant. Wrong target: cleaning recovers the constant gap between the two paths, not the mean, and r cancels.

10–13× worse — 341 false dispatches against 18.

A 3-day trailing median — fails the other way

A trailing median lags a rising quantity. Soiling climbs 0.240pp on a median day, so the estimate describes the plant as it was yesterday and reads too clean.

\$6,414 skipped on plant_1000; 24 plant-days; coverage 84.7% → 66.1%.

Both smoothed a quantity that does not need it. Behind the gate there is no noise left for a median to defend against — only lag for it to introduce. So s_0 is today's gated reading, unsmoothed.

Four things about that model I cannot yet defend

s_0 stands on a single day

One bad reading that clears the gate goes straight into a dispatch. Deleting the median removed the second line of defence.

Linear accumulation is asserted

Six plants hold a constant rate across both halves of their longest dry run, and nothing exceeds 11.74%, so saturation never binds here. A real plant saturates.

Recovery is modelled as instant

The formula assumes soiling drops to zero on the dispatch date. 10 of 12 plants need more than one crew-day; plant_1009 needs 6.3. Known false today.

T never decrements

Read as the expected wait of a memoryless process. Four of five regions agree; Atacama says 45 against a measured 80 on only three rain events.

3

What breaks first at 200 plants

Measured by generating the real thing — 200 plants × 5 years, 335,585 readings, through the same pipeline.

1

The interface, and it breaks long before 200

One table, grouped by region, every plant a row. No search, no filter, no sort, no pagination. The asset manager opens it to find 200 rows and 40 recommendations spread across three regions, and has to read all of them to find her day's work.

This is unusable at roughly 30 plants. It is the first thing to fail and the least technically interesting.

2

Crew capacity becomes the binding constraint — and nothing models it

Ranking answers “which plant”, never “which of these never gets done”. Atacama queues 13 plants and 72.9 crew-days against one crew and a 45-day rain horizon, so the crew reaches six before the rain arrives and the rest was never really on the list.

\$32,572 — 12% of ranked value — is stranded. Re-ranking by dollars per crew-day recovers only 2.2% of it.

3

The gate produces a triage queue nobody owns

Every withheld day is a plant that generated something unexplained, and the system deliberately refuses to say why. At twelve plants that is a curiosity; at 200 it is a daily work queue with no owner, no workflow and no way to close an item.

2.2 availability anomalies per day, every day — on 1,517 of 1,678 days, peaking at 7.

4

Not the database, which is the result I expected to go the other way

335,585 readings ingest and rank end to end in 4.3 s; the whole store is 93.4 MB; the fleet query is 0.22 ms. The PR band the gate depends on also stays empty at this scale — 0.3966 below, 0.8443 above.

SQLite is nowhere near a constraint, and the 8am burst is not a performance problem.

4

What I would build next, in order

Ordered by how much of the decision each one changes at 200 plants — not by how interesting it is to build.

1

An interface that survives 200 plants

Lead with a dispatch list, not a fleet table: the five plants worth a truck tomorrow, with the other 195 behind search, region and status filters. Region grouping is right at twelve and wrong at two hundred, because it forces a scan of everything to find anything.

Fixes the thing that breaks first, and needs no new modelling.

2

A capacity-aware scheduler

Crew calendars, multi-day jobs, and travel between sites, so the plan answers which jobs actually get done inside the rain horizon. Pair it with a per-region capacity report that names the binding crew, because the measurement says extra capacity beats smarter ordering.

Recovers the \$32,572 that re-ranking alone cannot — 2.2% against the rest.

3

A better soiling model

Per-plant accumulation rates rather than one median, saturation at high loss, partial washes below 8 mm, and recovery that starts when the crew finishes rather than when the truck leaves.

Sub-8 mm rain costs 9.4% in Andalusia and exactly 0% in Atacama — the correction is regional, and it already flips plant_1001.

4

Telemetry that separates a data fault from real dirt

Inverter status, curtailment signals, maintenance tickets and the 15-minute readings/ feed. Today the gate can say a reading is not credible as soiling; it can never say why, so it withholds the fault and the dirt together.

This is the ceiling on the whole system: 3,399 anomalies withheld at scale, every one unexplained.

5

What I would change about the brief

Ordered by what each one cost me.

Let the deck be longer than five slides

The brief asks for fifteen minutes covering architecture, a modelling decision, a scaling failure, a roadmap and a critique. Five slides forces either a thin deck or slides nobody can read. I would rather have eight and keep each one legible.

Supply data that identifies telemetry artefacts

Fault codes, inverter status, curtailment flags, maintenance windows. The quality gate is the load-bearing part of this system and it rests on one signal, PR, because that is all there is. With a second signal it could separate a metering failure from a real outage — and those two invert the commercial response.